

Transformers and Vision Transformer

Dr. Muhammad Sajjad

R.A: Kaleem Ullah

R.A: Ayaz Khan

Overview

- **Natural Language Processing (NLP)**
 - **Sentiment Analysis**
 - **History of Natural Language Processing**
 - **Different Approaches of NLP**
 - **Early Rule-Based Systems**
 - **Transition to Statistical Methods**
 - **Machine Learning in NLP**
 - **Deep Learning in NLP**
 - **Recurrent Neural Networks (RNNs) for NLP**
 - **Key Methods in NLP**
-
- **Transformer**
 - **Transformer Model Architecture**
 - **Transformer Model: Encoding and Decoding Components**
 - **Self-Attention: High-Level Overview**
 - **Advantages and Disadvantage of Transformer**
 - **Evolution of Dominant Architectures in Computer Vision**
-
- **Vision Transformers (ViTs)**
 - **ViTs Advantages, Challenges and Applications**
 - **Vision Transformer Architecture**
 - **Vision Transformers vs CNN**

Natural Language Processing (NLP)

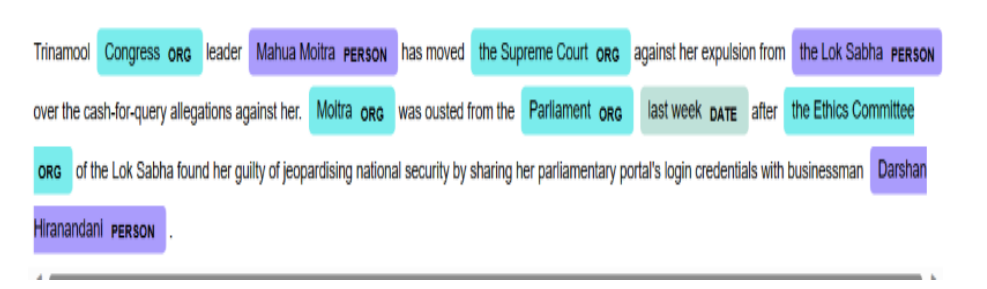
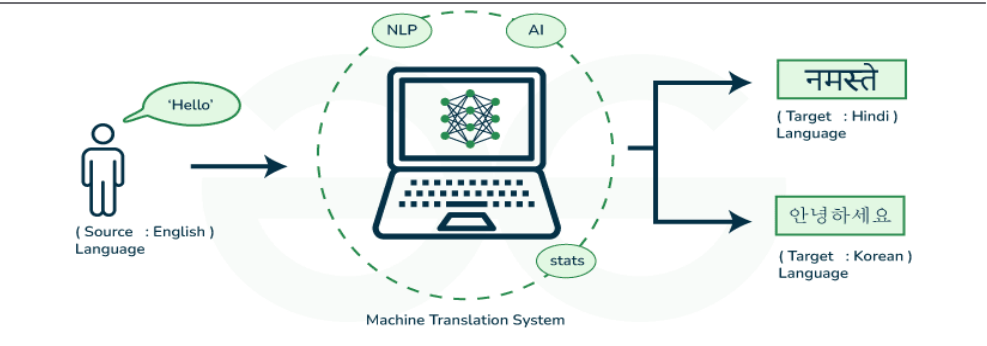
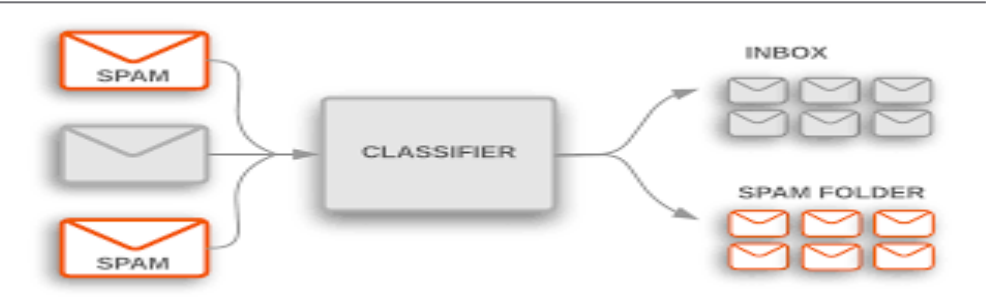
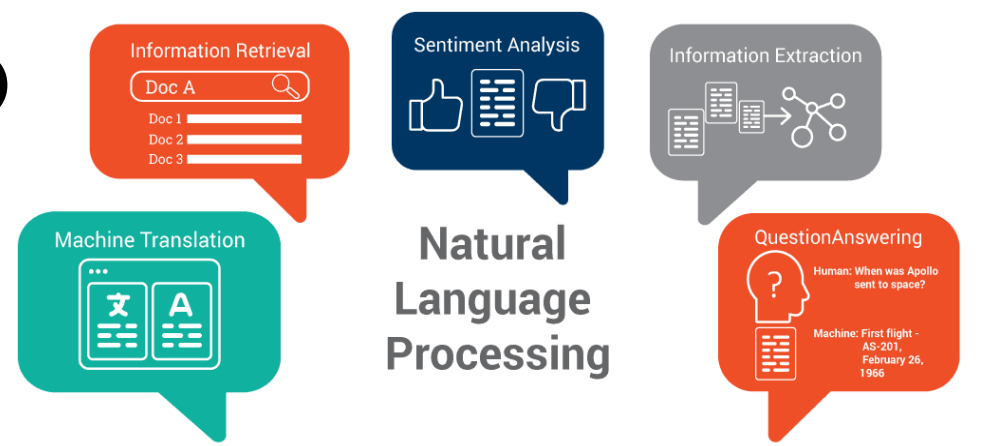
❑ **Natural Language Processing (NLP)** is a subfield of artificial intelligence that enables computers to understand, interpret, and generate human language.

❑ **Key Tasks in NLP**

- **Text Classification:** Categorizing text into predefined classes.
 - **Example:** Sorting customer emails into categories such as "inquiry," "complaint," and "feedback."
- **Sentiment Analysis:** Determining the sentiment expressed in text.
 - **Example:** Analyzing social media posts to identify whether the public sentiment towards a new product is positive, negative, or neutral.
- **Machine Translation:** Translating text from one language to another.
 - **Example:** Using Google Translate to convert a document from English to Spanish.
- **Named Entity Recognition (NER):** Identifying entities like names, dates, and organizations in text.
 - **Example:** Extracting names of people, locations, and dates from a news article.

❑ **Importance of NLP**

- **Real-World Applications:** NLP is used in chatbots, virtual assistants, language translation, and sentiment analysis.
- **Impact:** Improves human-computer interaction, automates text processing, and enhances accessibility.



Sentiment Analysis

❑ **Sentiment Analysis** is the process of using NLP to identify and categorize opinions expressed in text to determine whether the writer's attitude towards a particular topic, product, or service is positive, negative, or neutral.

❑ **Example of Sentiment Analysis:**

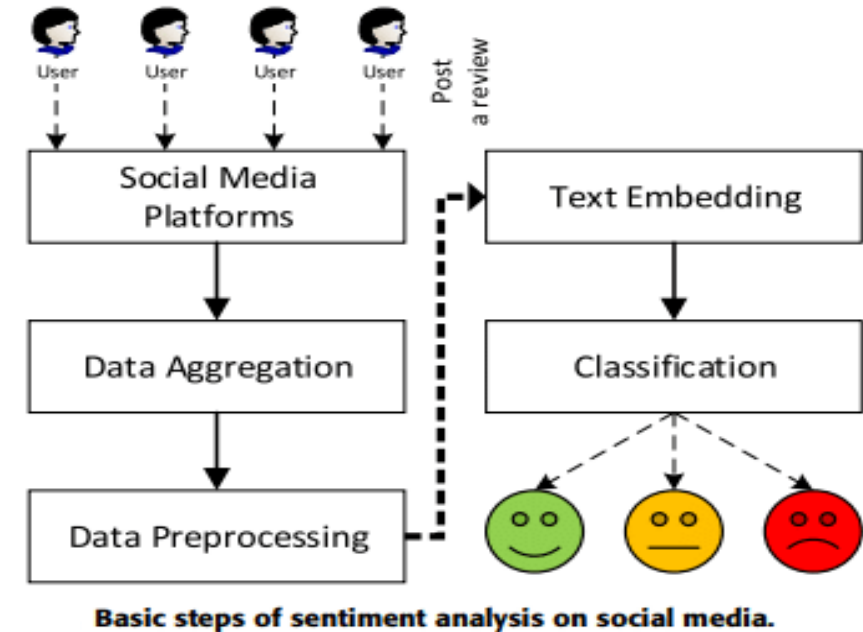
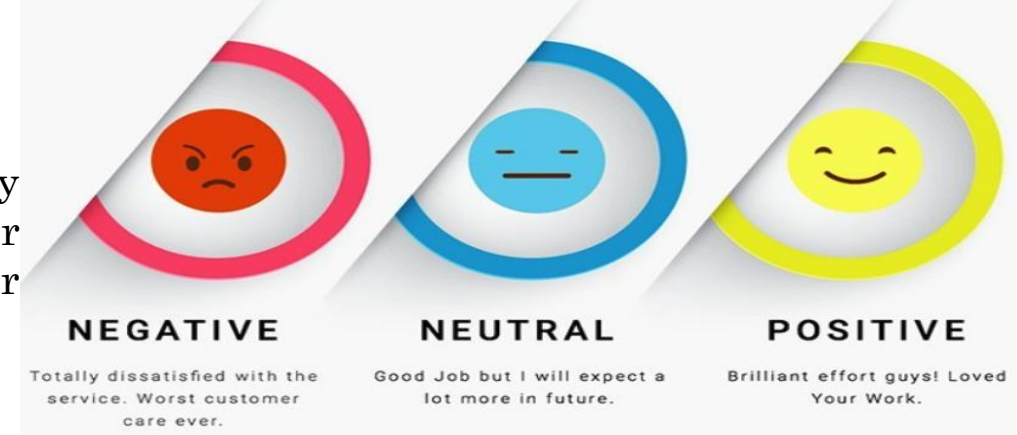
- **Example Sentence:** "I love the new design of this app, but I hate how it crashes frequently."
 - **Positive Sentiment:** "I love the new design of this app."
 - **Negative Sentiment:** "I hate how it crashes frequently."

❑ **Applications of Sentiment Analysis:**

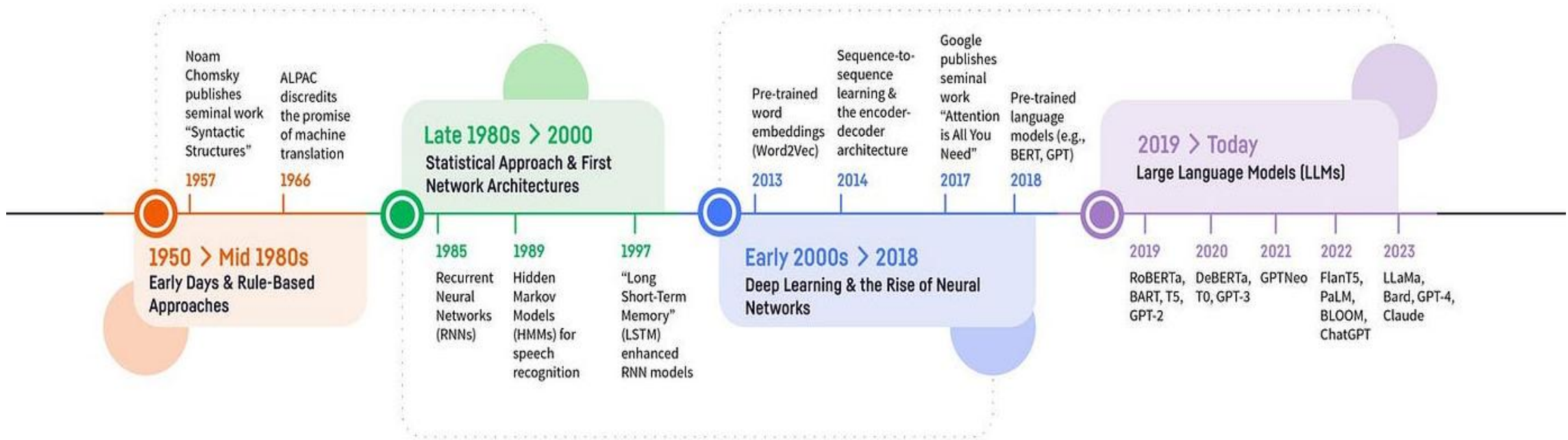
- **Customer Feedback Analysis:** Analyzing customer reviews and feedback to gauge satisfaction and areas for improvement.
- **Social Media Monitoring:** Monitoring social media platforms to understand public sentiment towards a brand or product.
- **Market Research:** Identifying consumer opinions and trends to inform product development and marketing strategies.

❑ **Use Case of Sentiment Analysis:**

- **Brand Management:** A company uses sentiment analysis to track public perception of their new product launch by analyzing tweets, reviews, and blog posts. The insights help the company to address negative feedback promptly and improve customer satisfaction.



History of Natural Language Processing (NLP)



Different Approaches to Natural Language Processing (NLP)

Early Rule-Based Systems: [\(Link\)](#)

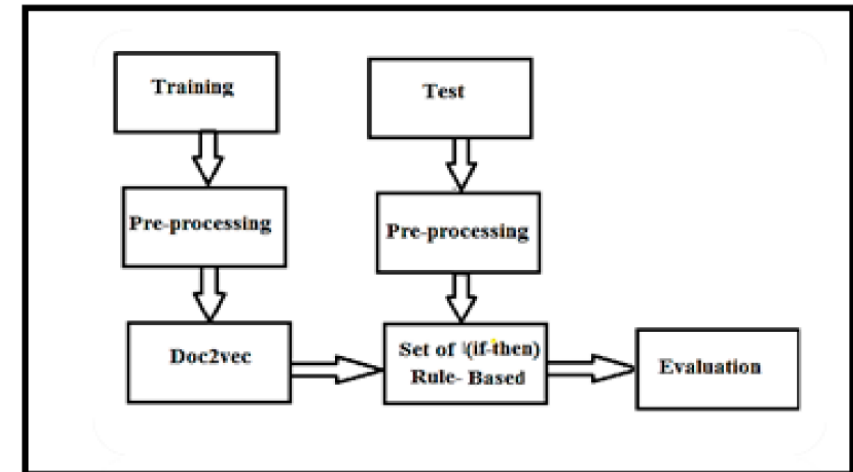
In the early days of NLP, systems were primarily rule-based. Linguists and programmers manually created a set of rules to process language.

❑ Examples:

- **Early Chatbots:**
 - **ELIZA:** Developed in the 1960s, used pattern matching and substitution to simulate conversation. ELIZA could emulate a Rogerian psychotherapist by matching user input against pre-defined patterns and providing scripted responses.
- **Language Parsers:**
 - **Example:** Early syntactic parsers used context-free grammars (CFGs) to analyze the structure of sentences.

❑ Applications:

- **Spell Checkers:** Early spell checkers used rule-based approaches to detect and suggest corrections for misspelled words.
- **Text Editors:** Grammar checking in word processors relied on predefined rules to highlight grammatical errors.



Different Approaches to Natural Language Processing (NLP)

Transition to Statistical Methods:

The 1980s and 1990s saw a shift from rule-based systems to statistical methods. This approach utilized large amounts of data and statistical techniques to understand and generate language.

❑ Examples:

- **Part-of-Speech Tagging:**

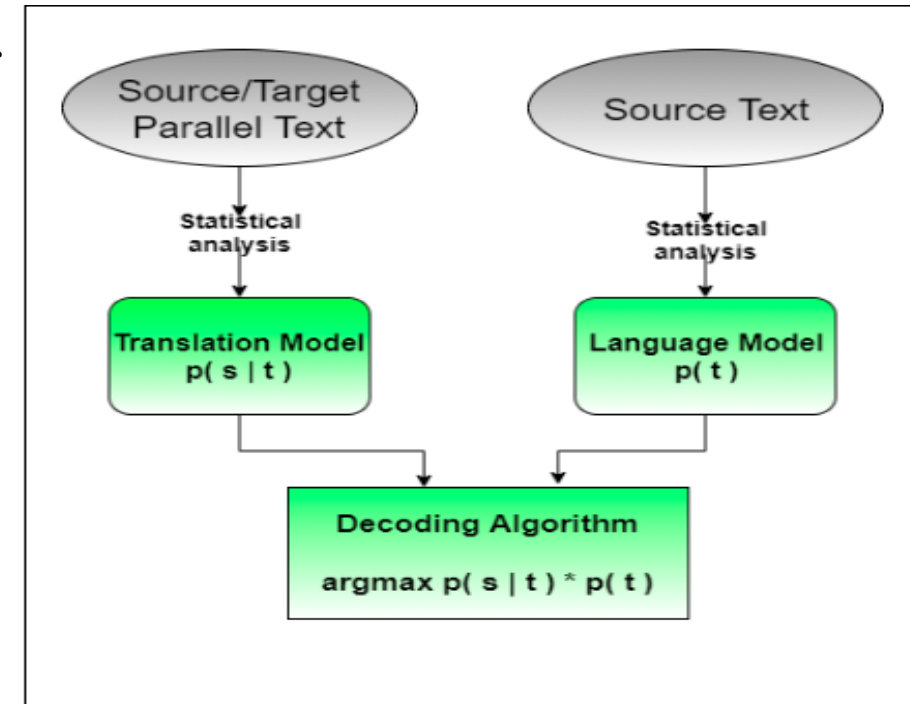
Hidden Markov Models (HMMs): Used probabilities to predict the correct tags for words in a sentence based on the context provided by surrounding words.

- **Speech Recognition:**

Example: Systems like IBM's Tangora used statistical models to improve the accuracy of speech-to-text conversion.

❑ Applications:

Machine Translation: Statistical methods enabled more accurate translation systems, such as Google's early translation algorithms which relied on large corpora of bilingual text.



Different Approaches to Natural Language Processing (NLP)

Machine learning in NLP

- The late 1990s and early 2000s marked the introduction of machine learning in NLP. Machine learning models, such as decision trees and support vector machines, were trained on labeled datasets to perform various NLP tasks.

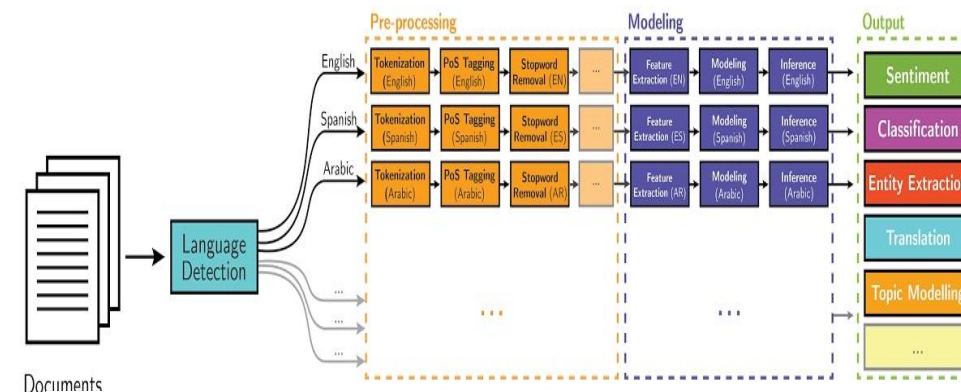
Example:

Named Entity Recognition (NER):

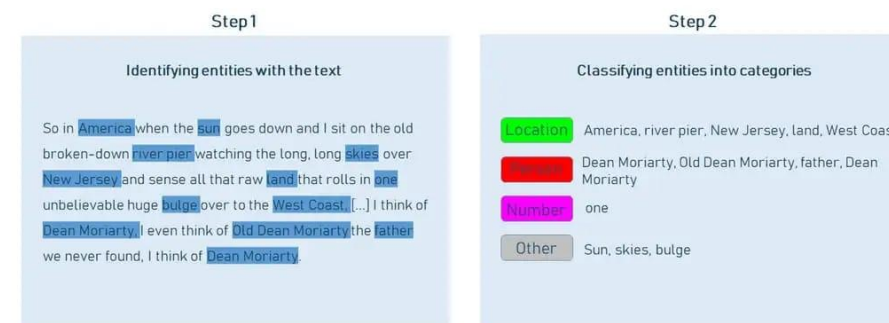
- Machine learning models trained on annotated corpora to identify entities in text, such as names of people, organizations, locations, dates, and other specific types of information.
- Example:** Identifying entities in a news article to extract information like "Barack Obama," "White House," and "January 20, 2009."

Applications:

- Text Classification:** Assigning predefined categories to text documents.
 - Example:** Classifying emails into "spam" and "non-spam" categories.
- Sentiment Analysis:** Determining the sentiment expressed in a piece of text.
 - Example:** Analyzing customer reviews to determine whether the feedback is positive, negative, or neutral.



TWO STEPS OF THE NAMED ENTITY RECOGNITION PROCESS



Different Approaches to Natural Language Processing (NLP)

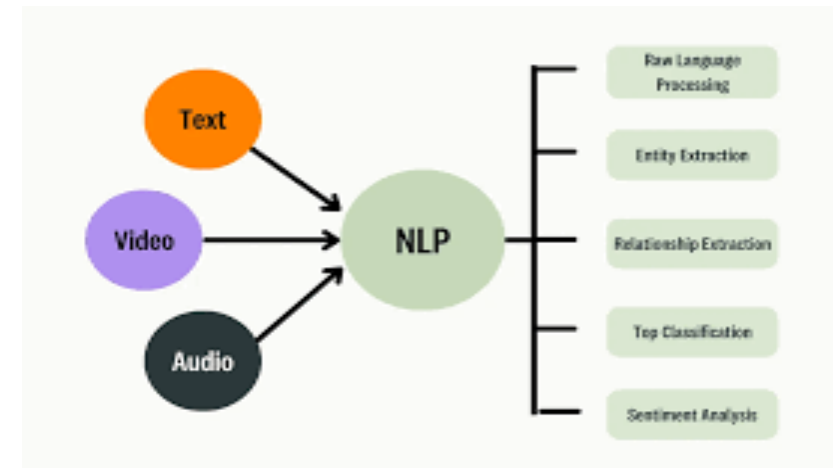
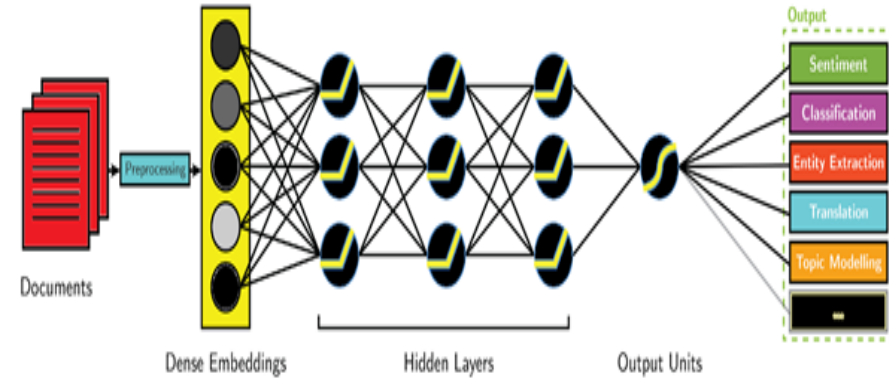
Deep Learning in Natural Language Processing (NLP)

- Deep learning has revolutionized NLP by leveraging neural networks, including recurrent neural networks (RNNs), convolutional neural networks (CNNs), and transformers.
- These models have significantly enhanced the performance of various NLP tasks by learning from extensive datasets and capturing intricate patterns in text.
- Deep learning methods allow NLP systems to understand and generate human-like language, enabling advancements in areas such as machine translation, sentiment analysis, text summarization, and more.

❑ Language Models:

- **GPT-3:** A transformer-based model capable of generating human-like text, performing translation, and answering questions. It has been used in various applications, including content generation, chatbots, and language understanding tasks.
- **BERT (Bidirectional Encoder Representations from Transformers):** Another influential transformer-based model known for its pre-training approach, which enables fine-tuning for a wide range of NLP tasks, such as question answering, sentiment analysis, and named entity recognition.

- ❑ **Text Generation: Example:** Utilizing GPT-3 to compose articles, craft poetry, or simulate conversations. Similarly, models like GPT-2 and BERT can be used for text generation tasks, demonstrating the versatility of deep learning in creative and practical applications alike.



Recurrent Neural Networks (RNNs) For NLP

- Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data by maintaining a hidden state that captures information from previous inputs. In Natural Language Processing (NLP), RNNs have been widely used for tasks requiring sequence modeling, such as language modeling, machine translation, and sentiment analysis.

❑ For Example:

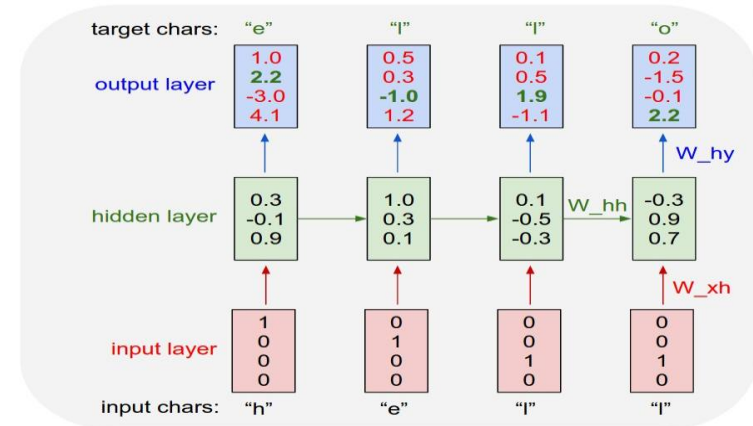
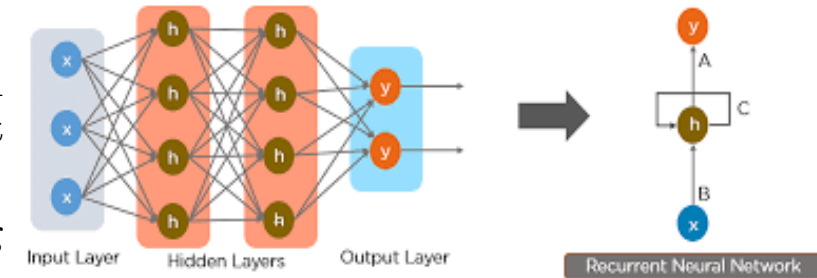
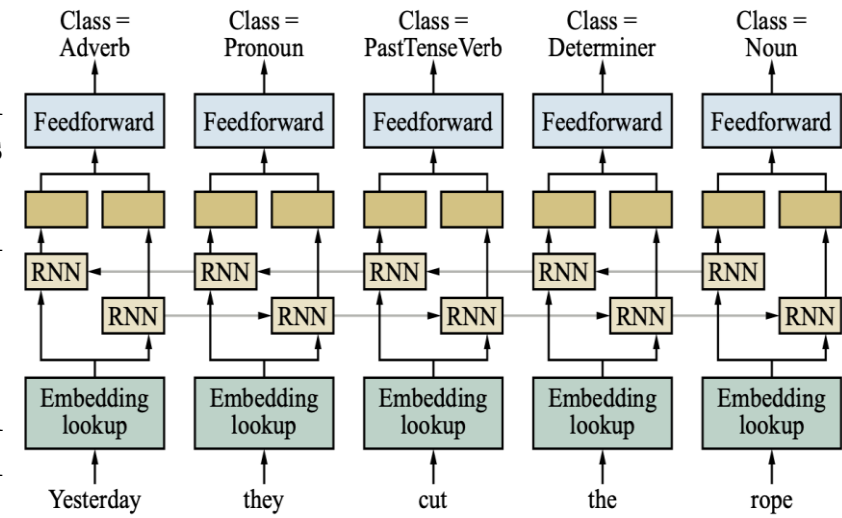
- Language Modeling:** RNNs can predict the next word in a sequence given the previous words, which is fundamental for tasks like speech recognition and autocomplete.

❑ Applications:

- Machine Translation:** RNNs have been employed in machine translation systems to translate text from one language to another by processing input sequences and generating output sequences.
- Sentiment Analysis:** RNNs can analyze sentiment in text by processing sequences of words and classifying the sentiment expressed.

❑ Limitations:

- Vanishing Gradient Problem:** RNNs often face difficulties in capturing long-term dependencies in sequences due to the vanishing gradient problem, where gradients become extremely small during backpropagation, hindering the learning process.
- Limited Contextual Understanding:** RNNs have a fixed-size hidden state, which limits their ability to capture extensive contextual information from long sequences, impacting tasks that require understanding of broader contexts.



Key Methods in Natural Language Processing (NLP)

- Several key methods are fundamental to Natural Language Processing (NLP), enabling tasks such as language understanding, text generation, and sentiment analysis. These methods play a crucial role in processing and representing textual data for machine learning models.

Tokenization:

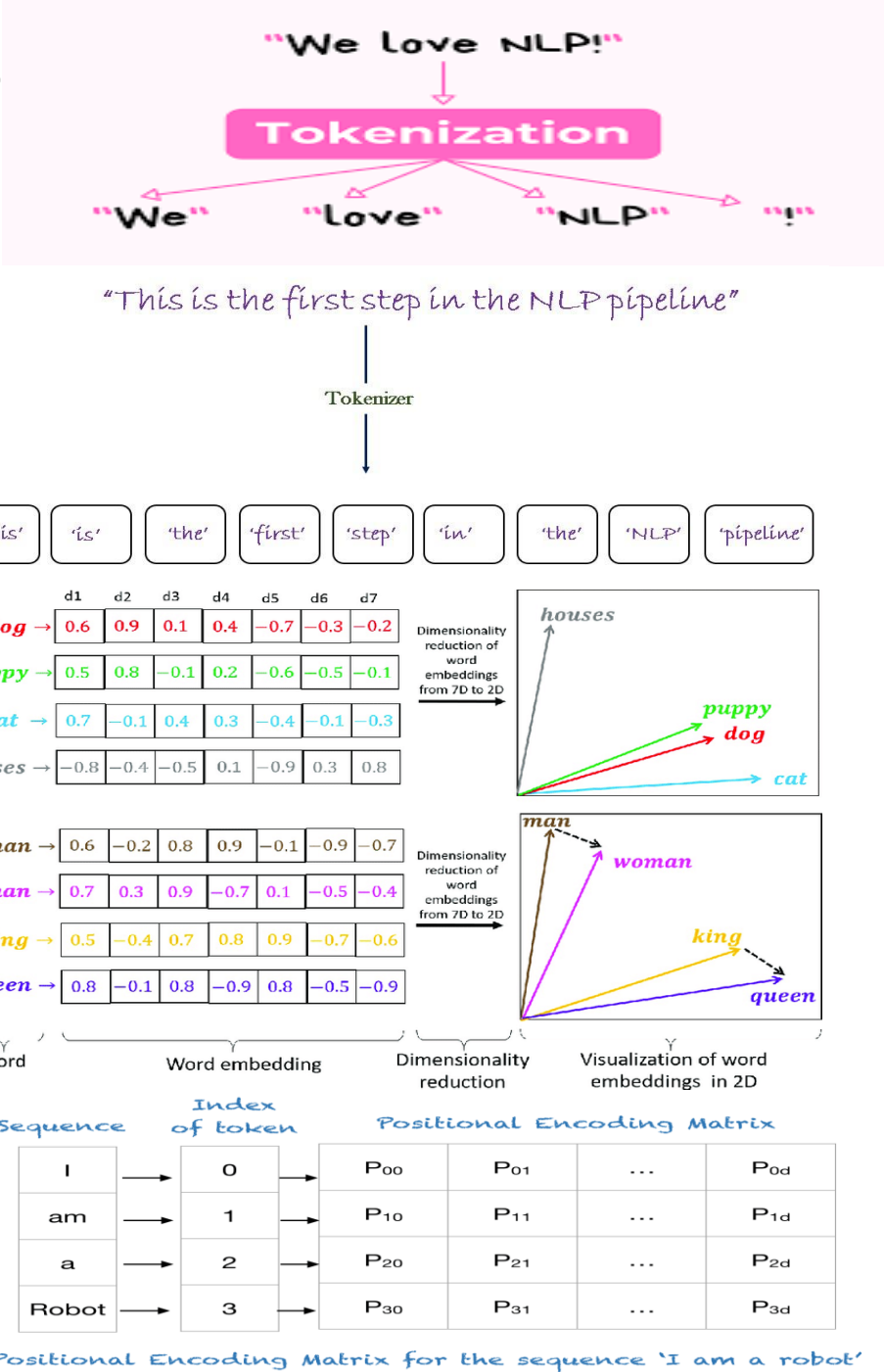
- Tokenization involves breaking down a piece of text into smaller units, such as words or subwords, known as tokens. This process facilitates further analysis and processing of text data.
- Example:** Sentence: "The quick brown fox jumps over the lazy dog." Tokens: ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog", "."]

Word Embedding:

- Word embedding is a technique used to represent words as dense vectors in a continuous vector space. This representation captures semantic relationships between words, enabling models to understand and generalize from textual data.
- Example:** Word: "cat" Vector Representation: [0.2, -0.5, 0.8, ...]

Positional Embedding:

- Positional embedding is used in sequence-to-sequence models, such as transformers, to encode the positional information of tokens in a sequence. This allows models to differentiate between the positions of words in a sentence.
- Example:** Token: "fox" Positional Embedding: [0.3, 0.1, -0.2, ...]



Transformer

Transformer

- The Transformer model, introduced by Vaswani et al. in the paper “Attention is All You Need” (2017), is a deep learning architecture that has revolutionized Natural Language Processing (NLP). Unlike traditional recurrent models like RNNs and LSTMs, the Transformer relies entirely on self-attention mechanisms to process sequences of data, enabling parallelization and improved handling of long-range dependencies.

Applications:

- **Language Modeling:** Transformers have been used to build powerful language models such as GPT-3 and BERT, which excel in tasks like text generation, translation, and question answering.
- **Machine Translation:** The Transformer model has set new benchmarks in machine translation tasks, outperforming previous models like LSTMs and GRUs.
- **Text Summarization:** Transformers are effective in summarizing long documents by capturing the most relevant information and generating concise summaries.
- **Sentiment Analysis:** Transformers can analyze the sentiment expressed in text by understanding the contextual meaning of words and phrases, leading to accurate sentiment classification.
- **Speech Recognition:** Transformers have been applied to speech recognition tasks, improving the accuracy of transcribing spoken language into written text.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

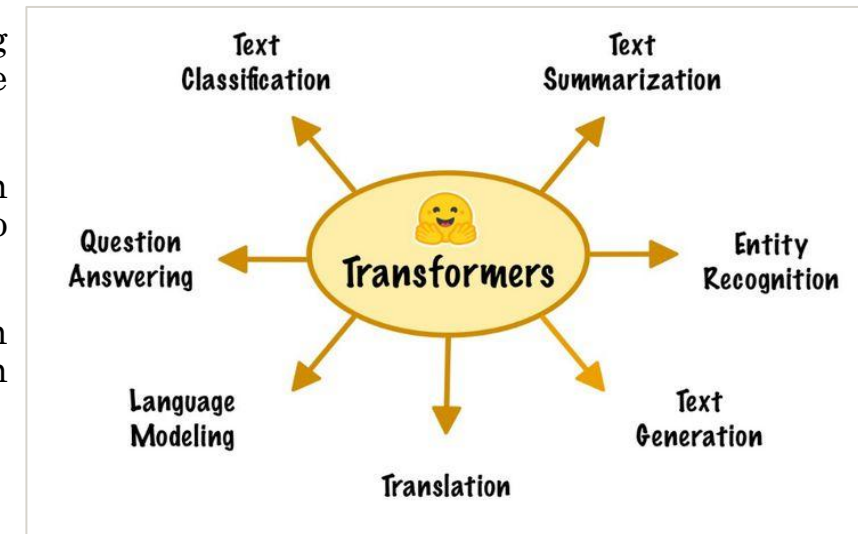
Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

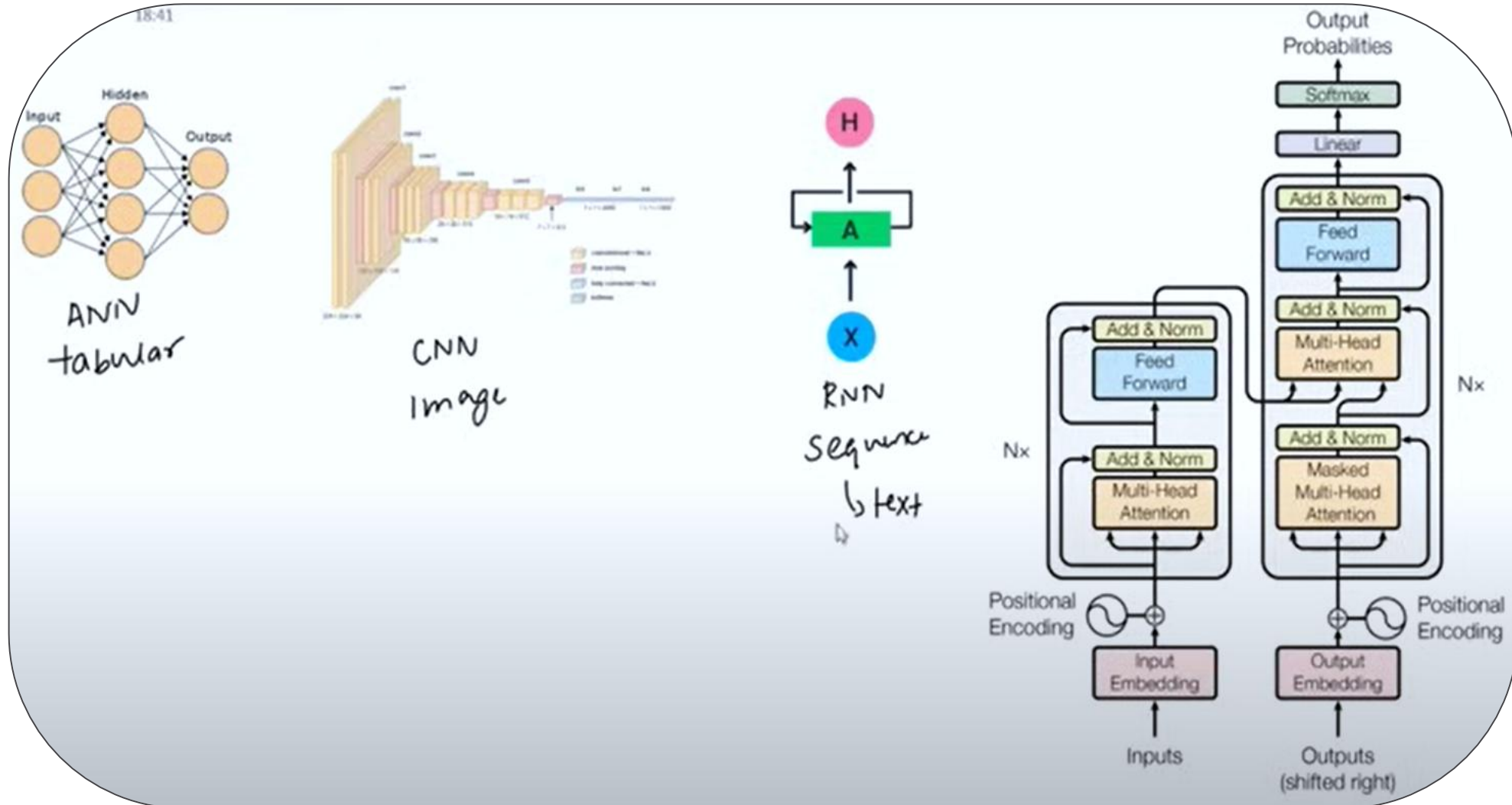
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best



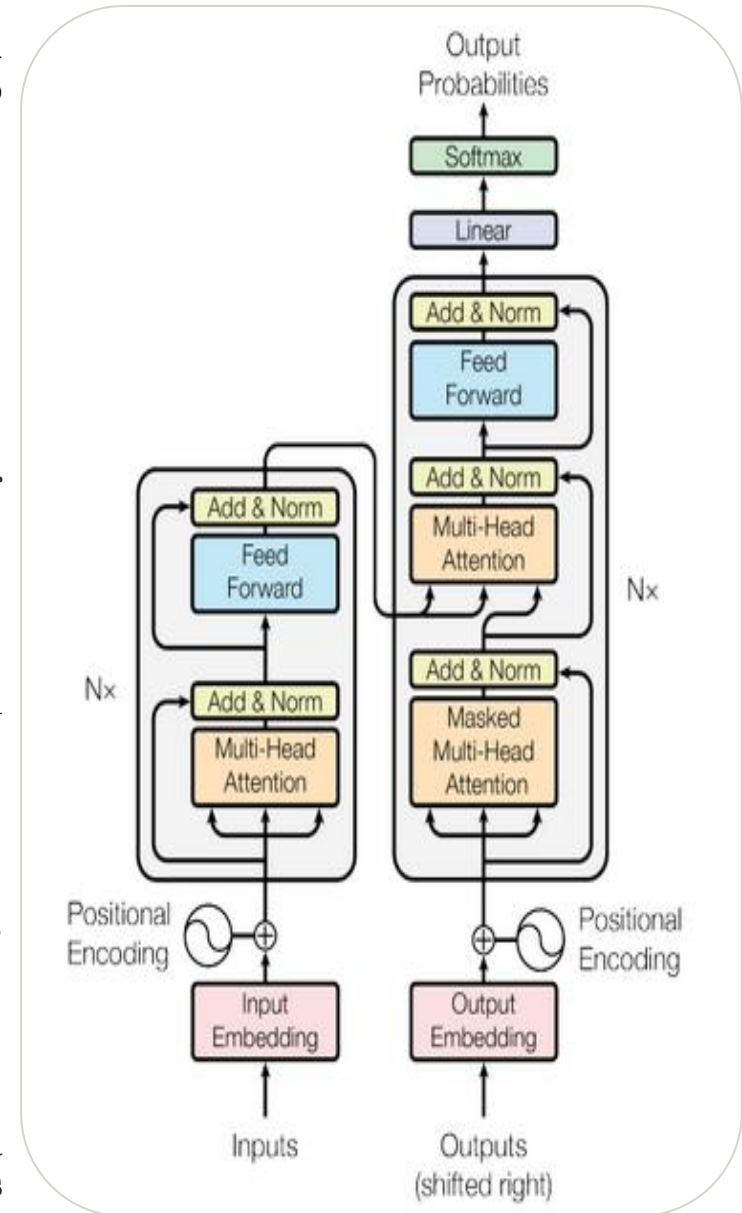
Neural Network Architectures for Different Data Types

Different neural network architectures are designed to handle specific types of data effectively.



Transformer Model Architecture

- The Transformer model architecture is designed to handle sequential data through a series of key components that work together to process and transform inputs into outputs efficiently. Here's a look at its main components:
- ❑ **Input Embedding:**
 - Converts input tokens into dense vector representations.
 - These embeddings capture the semantic meaning of the words.
 - ❑ **Positional Encoding:**
 - Adds information about the position of each token in the sequence.
 - Ensures the model takes the order of tokens into account, which is crucial for understanding context.
 - ❑ **Self-Attention:**
 - Allows each token to attend to every other token in the sequence.
 - Helps the model to weigh the importance of different tokens when processing each token, capturing dependencies regardless of their distance in the sequence.
 - ❑ **Multi-Head Attention:**
 - Consists of multiple self-attention mechanisms running in parallel.
 - Enables the model to focus on different parts of the sequence simultaneously, capturing a richer set of dependencies.
 - ❑ **Normalization and Feed-Forward Network:**
 - **Normalization:** Layer normalization is applied to stabilize and speed up training.
 - **Feed-Forward Network:** After normalization, the output is passed through a position-wise feed-forward network for further transformation. This typically involves two linear layers with a ReLU activation in between.



The Transformer – model architecture.

Transformer Model Architecture

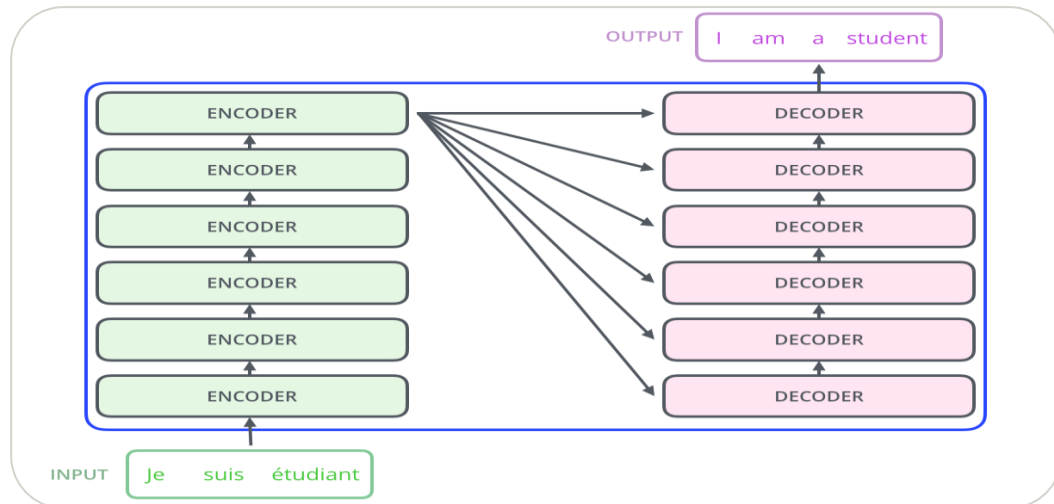
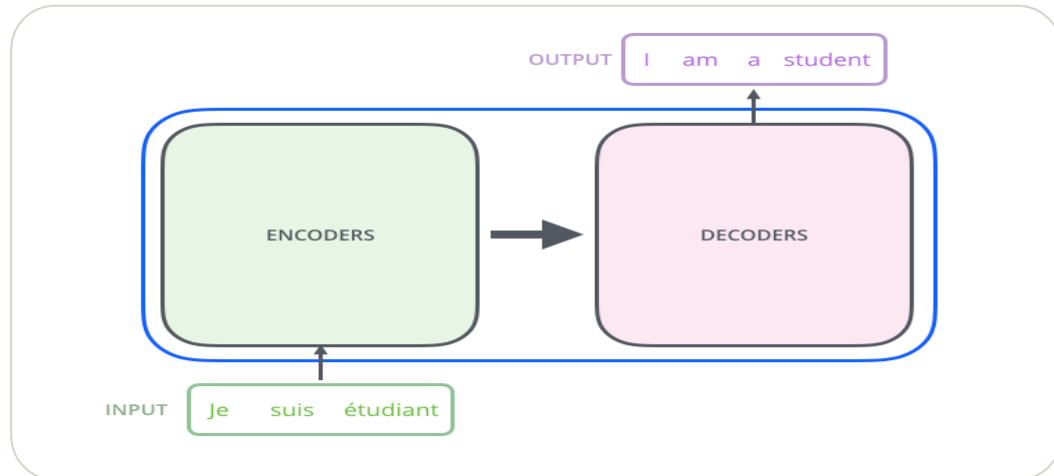
- Let's begin by looking with the model as a black box that translates a sentence from one language to another.
- Diagram shows translating input text to output text, highlighting encoding and decoding components with arrows showing data flow.

❑ Encoding Component:

- Transforms input sentence into vectors representing semantic content of the text.
- Consists of multiple stacked encoders, typically six, but adjustable based on application needs.

❑ Decoding Component:

- Converts encoded vectors into a translated sentence.
- The decoding component mirrors the encoding component with the same number of decoders stacked on top of each other.
- Each decoder layer also includes additional mechanisms to process the encoded information and generate the final output.



Transformer Model: Encoding and Decoding Components

❑ Encoder Structure:

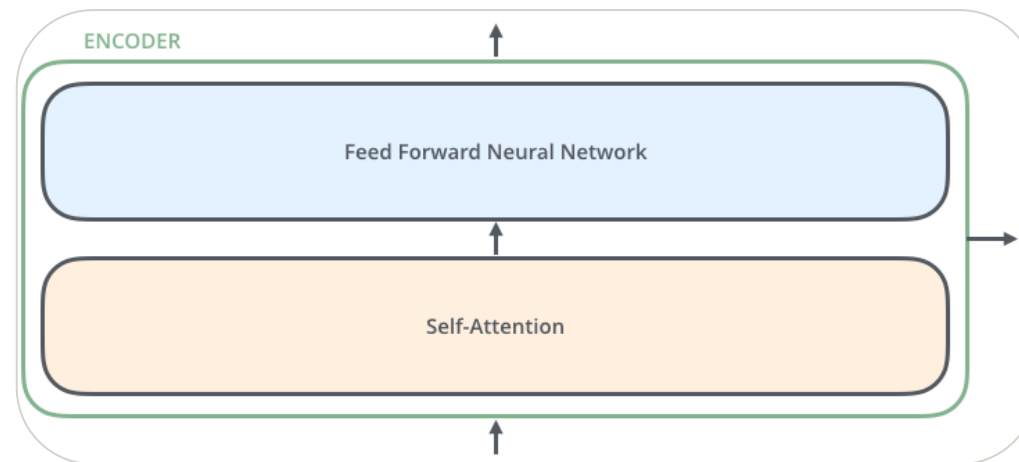
Each encoder is broken down into two main sub-layers:

- **Self-Attention Layer:**

- Helps the encoder look at other words in the input sentence while encoding a specific word.
- Captures dependencies and relationships between words.

- **Feed-Forward Neural Network:**

- Outputs of the self-attention layer are fed into this network.
- The exact same network is independently applied to each position.
- Transforms the data further.



❑ Decoder Structure:

The decoder has both those layers, but between them is an attention layer that helps the decoder focus on relevant parts of the input sentence.

- **Self-Attention Layer:**

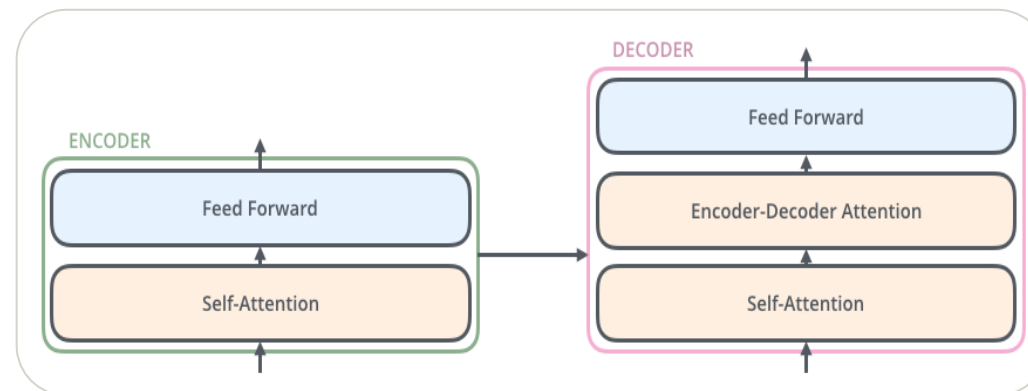
- Allows the decoder to attend to different positions in the input sequence of the target language.

- **Encoder-Decoder Attention Layer:**

- Helps the decoder focus on relevant parts of the input sentence.
- Similar to the attention mechanism in seq2seq models.

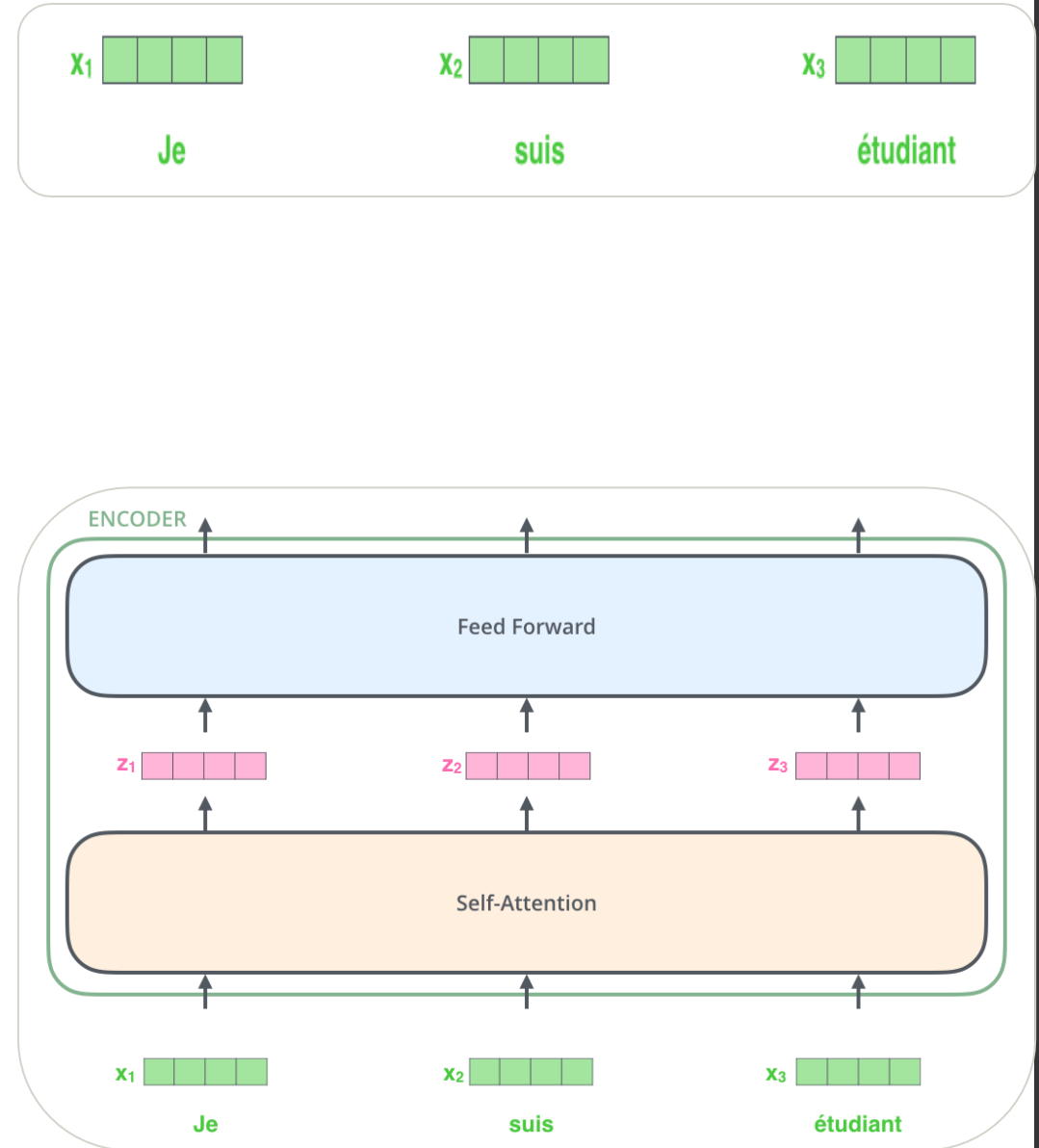
- **Feed-Forward Neural Network:**

- Similar to the encoder, it transforms the data at each position.



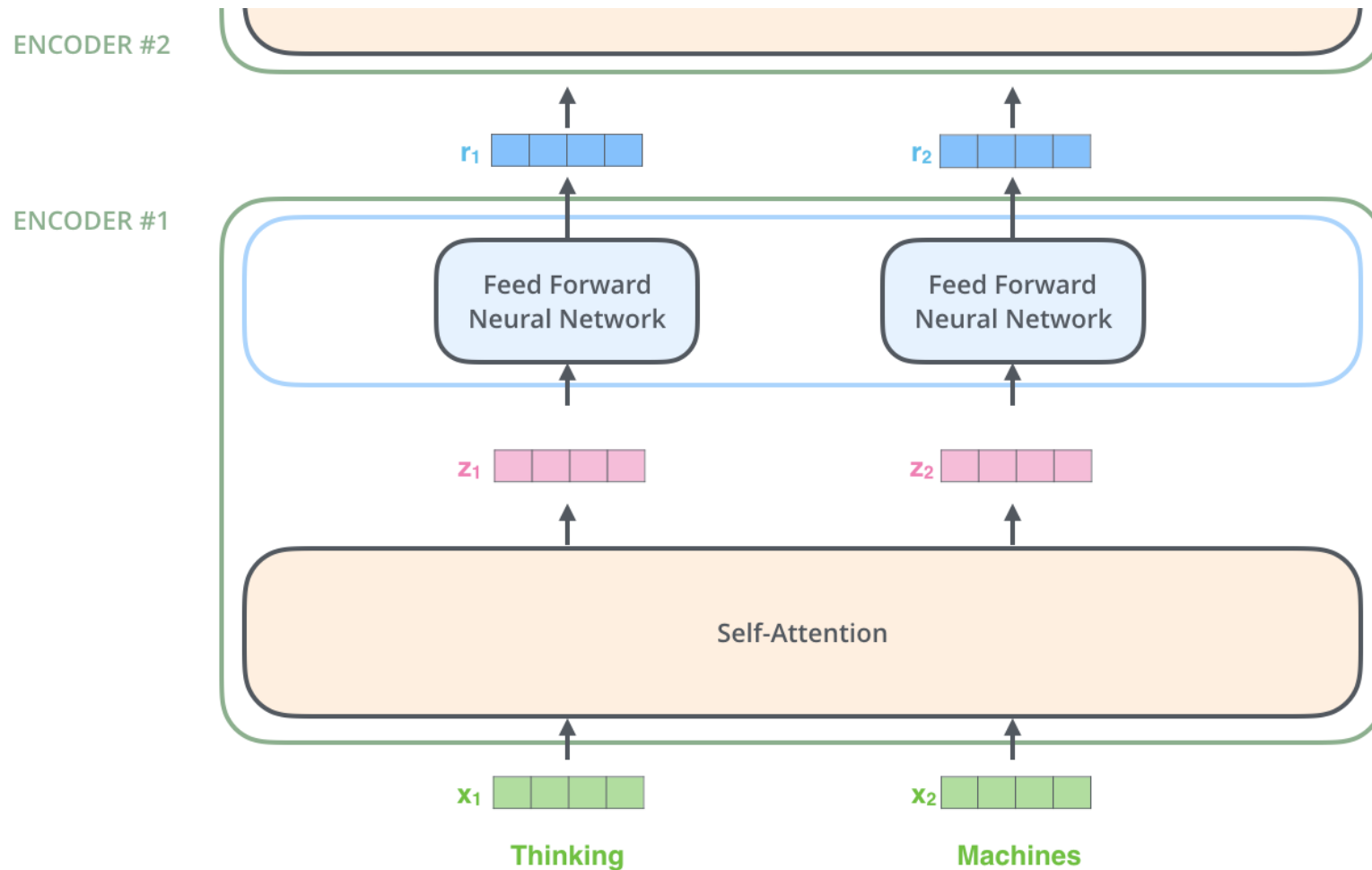
Bringing The Tensors Into The Picture

- Let's start to look at the flow of various vectors/tensors through model components to transform input into output.
- Converting each input word into a vector using an embedding algorithm.
- Embedding occurs in the bottom-most encoder, which converts words into 512-size vectors.
- All encoders receive 512-size vectors: word embeddings in the bottom encoder, and outputs from the encoder below in others.
- The list size is a set hyperparameter, typically the length of the longest sentence in the training dataset.
- Words in the input sequence pass through each of the two layers of the encoder.
- In the Transformer, each word flows through its own path in the encoder.
- There are dependencies between these paths in the self-attention layer, but the feed-forward layer does not have, allowing parallel execution.



Encoding:

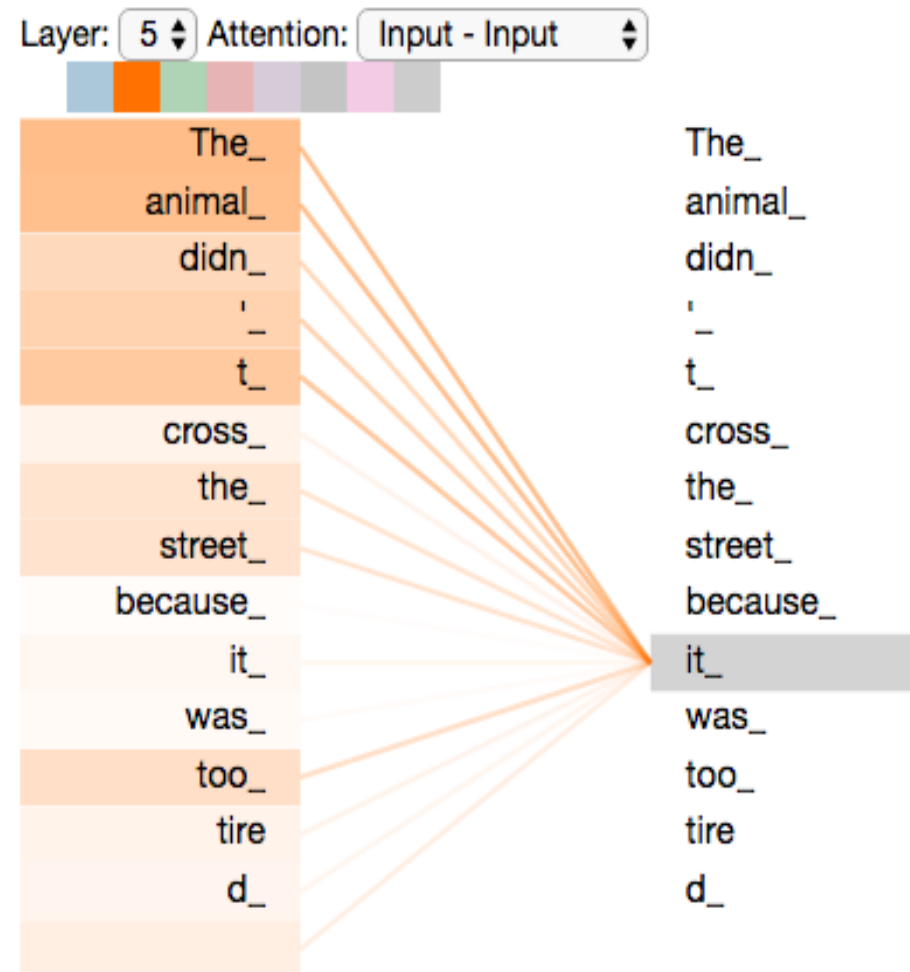
An encoder receives a list of vectors as input. It processes this list by passing these vectors into a 'self-attention' layer, then into a feed-forward neural network, then sends out the output upwards to the next encoder.



Self-Attention: High-Level Overview

Self-attention helps models understand word relationships in a sentence.

- Example:
The animal didn't cross the street because it was too tired.
- Goal:
Determine if “it” refers to “animal” or “street”.
- Self-Attention associates “it” with “animal” in the given example.
- Allows models to consider all words in a sentence for better context.
- Unlike RNNs that rely on hidden states, self-attention looks at all words simultaneously.
- Essential for accurate sentence encoding and translation in Transformers.



Transformer Model: Encoding and Decoding Components

❑ Input Embedding:

- Embedding is a fundamental concept in NLP that involves representing words or tokens as dense vectors in a continuous vector space.
- In Transformers, embedding plays a crucial role in converting discrete tokens (words or sub words) into continuous vector representations that capture their semantic meaning.

❑ Function of Embedding in Transformers:

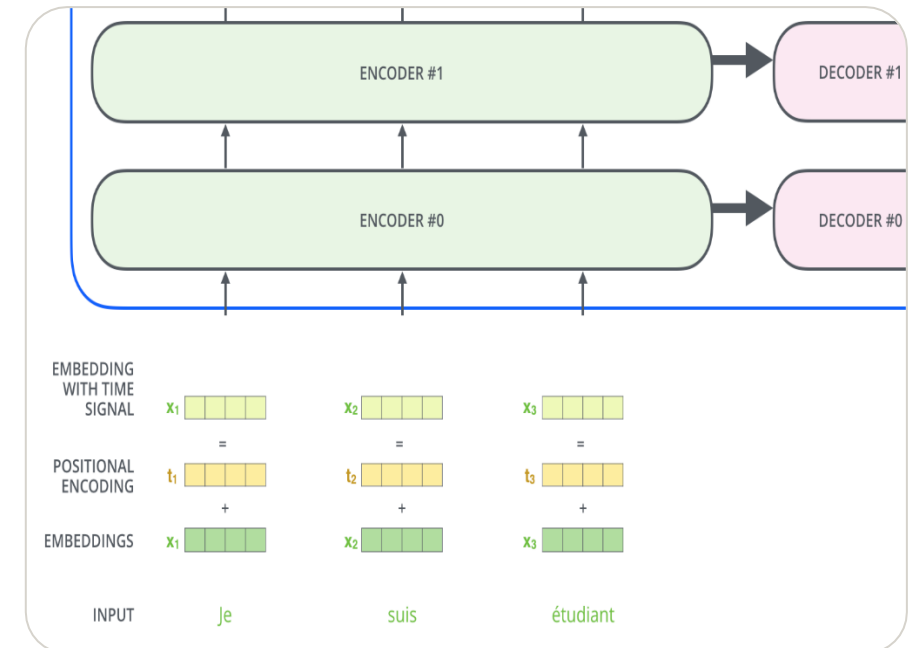
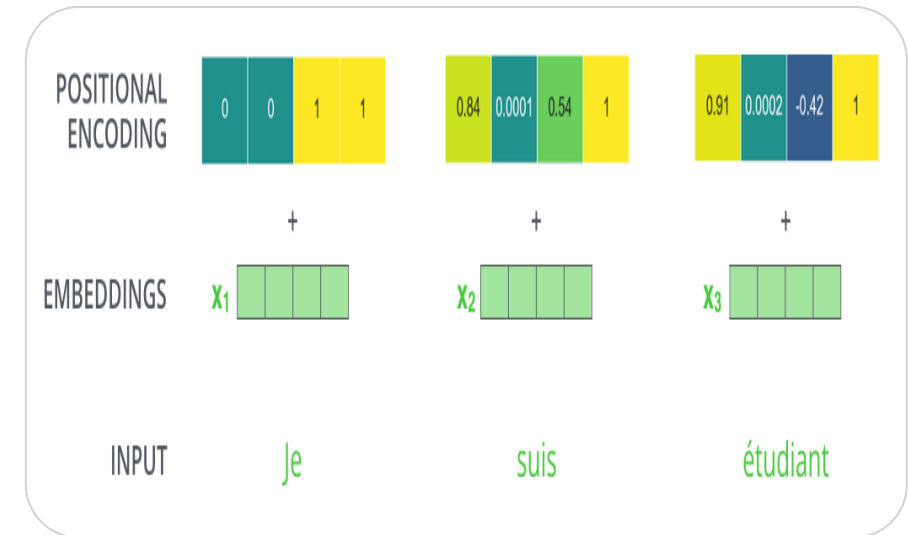
- The embedding layer in Transformers converts each token in the input sequence into a fixed-size dense vector representation.
- These dense vectors encode semantic information about the tokens, enabling the model to understand and process textual data effectively

❑ Need for Positional Encoding:

- One thing that's missing from the model as we have described it so far is a way to account for the order of the words in the input sequence.

❑ Solution: Positional Encoding

- The transformer adds a vector to each input embedding.
- These vectors follow a specific pattern that the model learns, which helps it determine the position of each word, or the distance between different words in the sequence.
- The intuition is that adding these values to the embeddings provides meaningful distances between the embedding vectors once they're projected into Q/K/V vectors and during dot-product attention.



Transformer Model: Encoding and Decoding Components

❑ Encoder Architecture:

- In the architecture of the encoder in Transformers, each sub-layer (such as self-attention and feed-forward neural network) has a residual connection around it.
- Additionally, each sub-layer is followed by a layer-normalization step.

❑ Residual Connection:

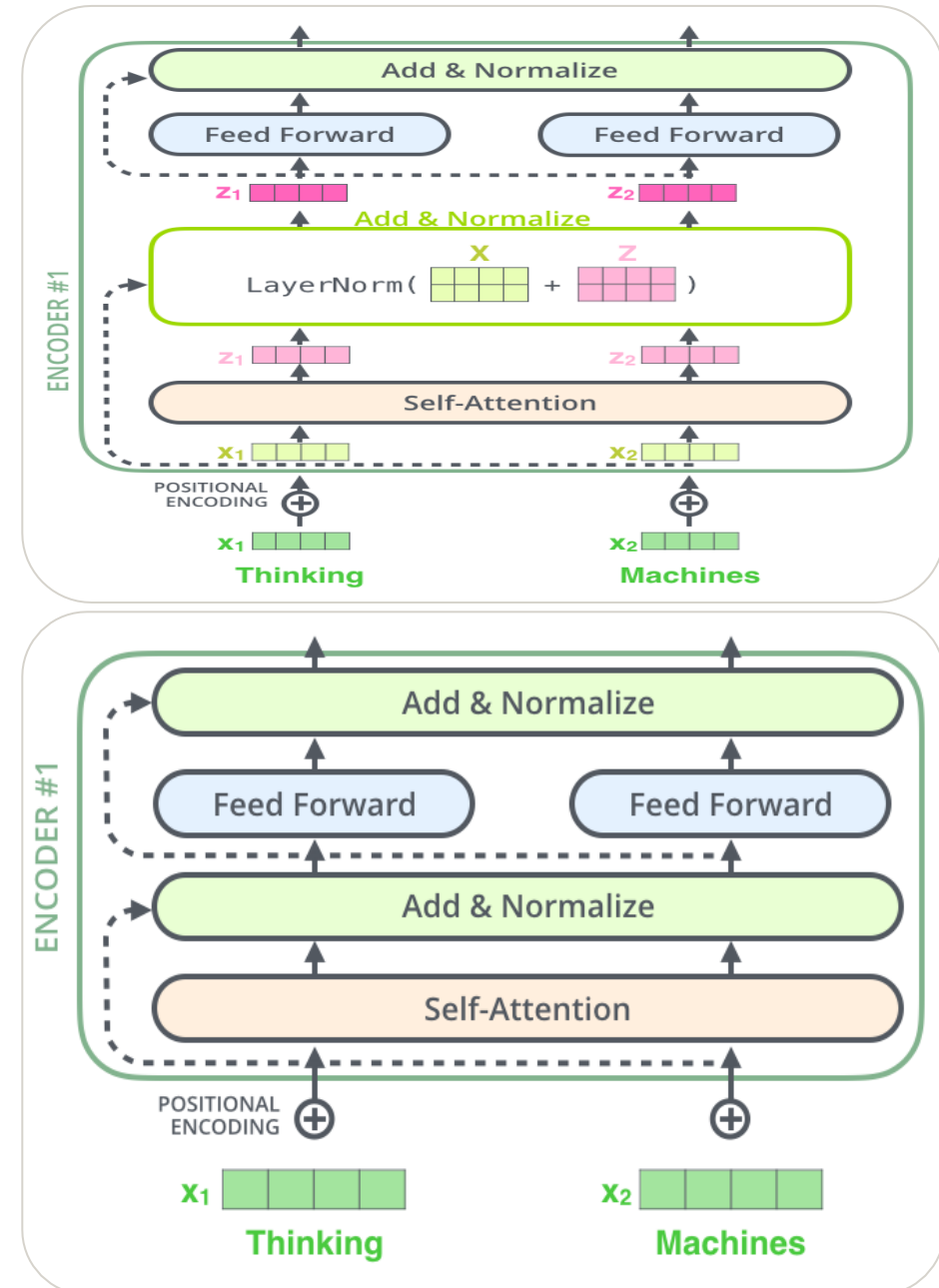
- A residual connection allows the original input to bypass the sub-layer and be directly added to the output of the sub-layer.
- This helps in mitigating the vanishing gradient problem and facilitates smoother gradient flow during training.

❑ Layer Normalization:

- Layer normalization is applied after each sub-layer to stabilize and speed up training.
- It normalizes the activations of the sub-layer across the feature dimension.

❑ Benefits:

- Residual connections and layer normalization help in improving the stability and efficiency of training Transformer models.
- They contribute to smoother optimization and faster convergence during training.



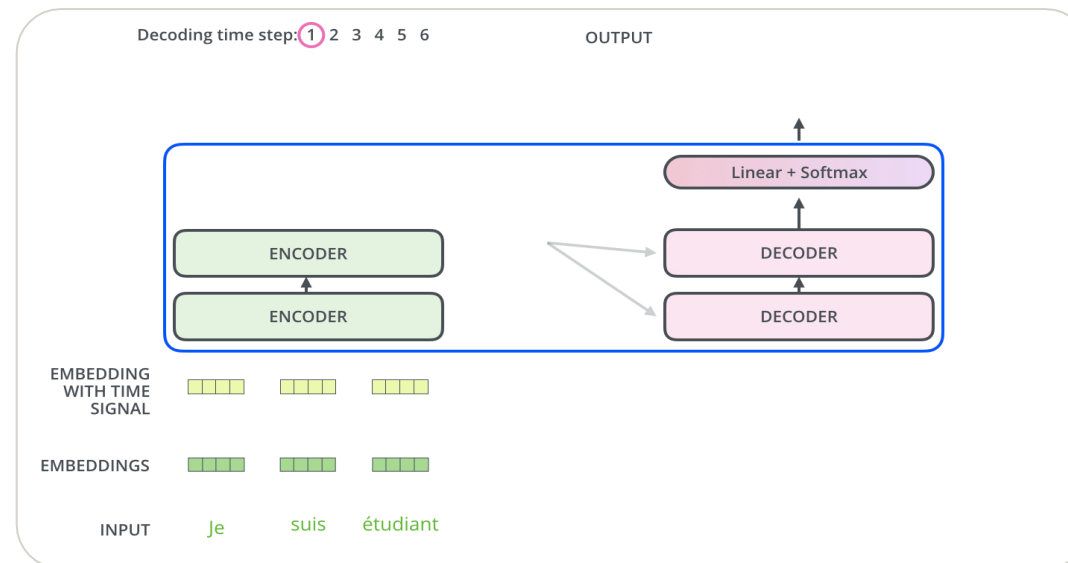
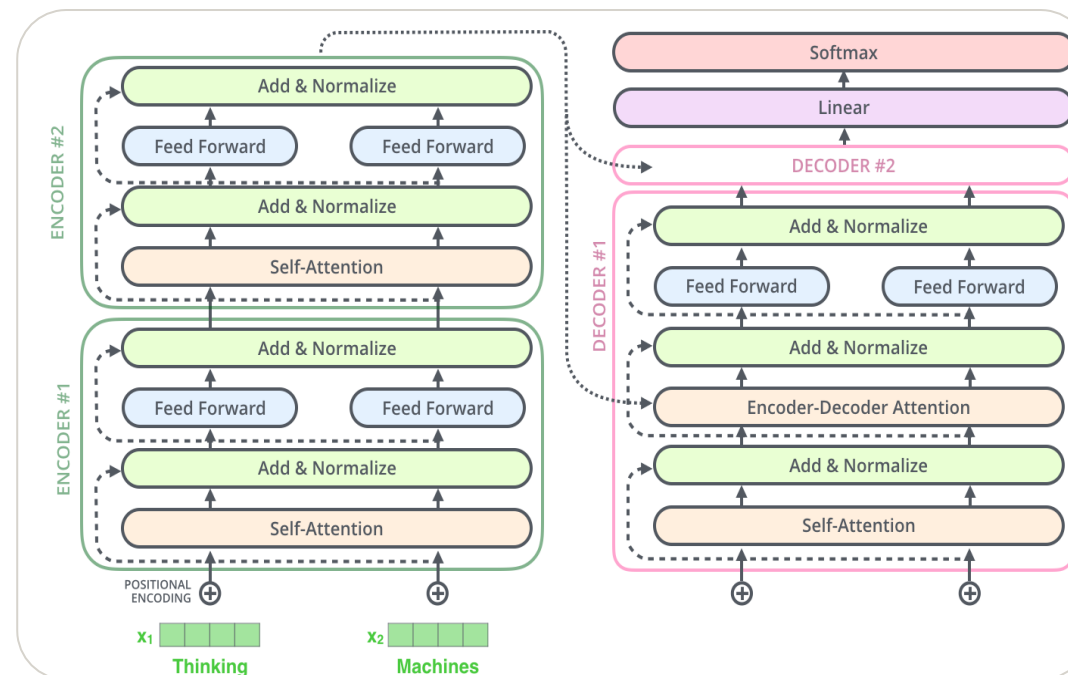
Transformer Model: Encoding and Decoding Components

❑ Decoder Architecture:

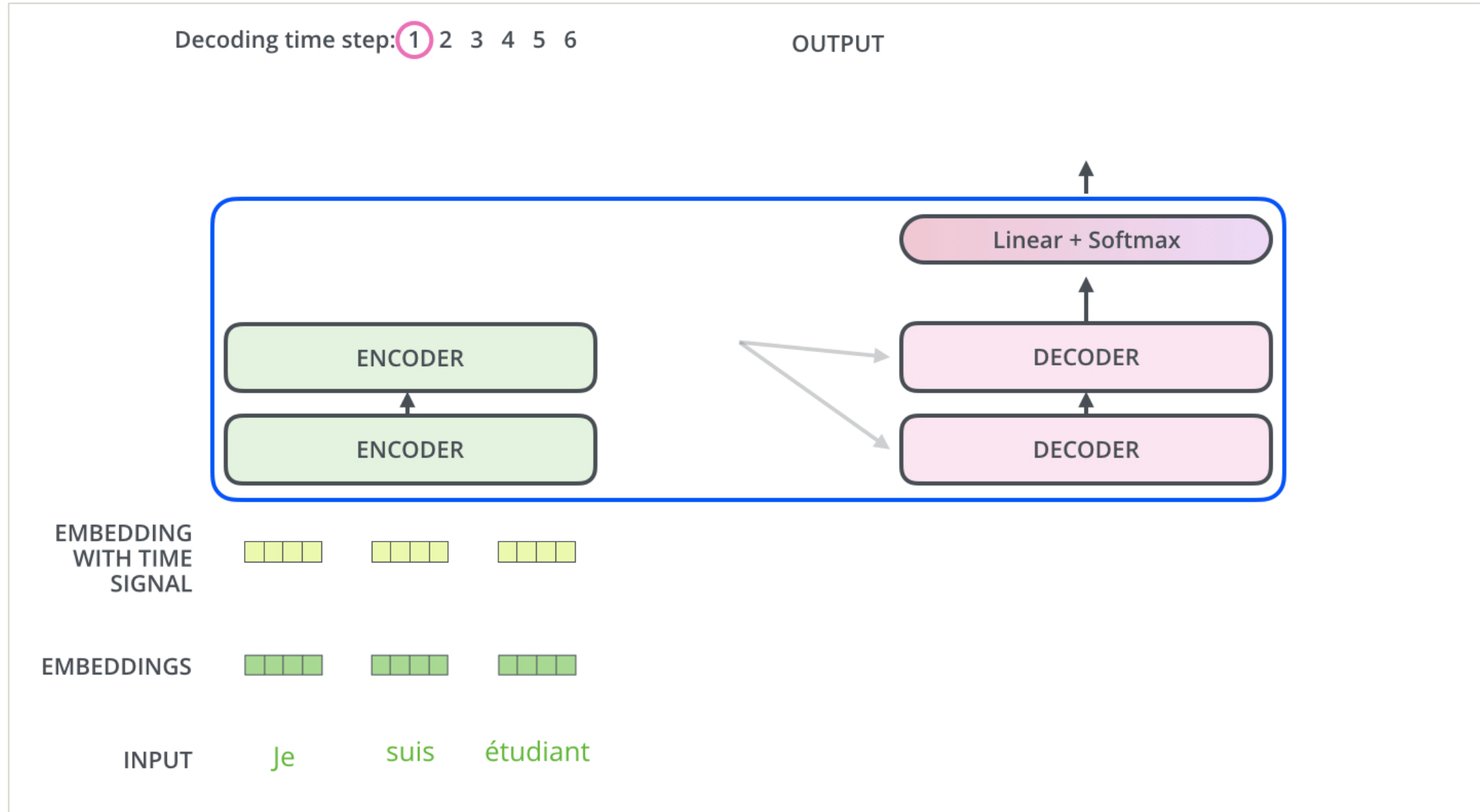
- Similar to the encoder, the decoder in Transformers also consists of multiple stacked layers.
- Each layer in the decoder has sub-layers, including self-attention and encoder-decoder attention, followed by feed-forward neural networks.
- Each sub-layer in the decoder includes a residual connection and layer normalization.

❑ Functioning of Decoder:

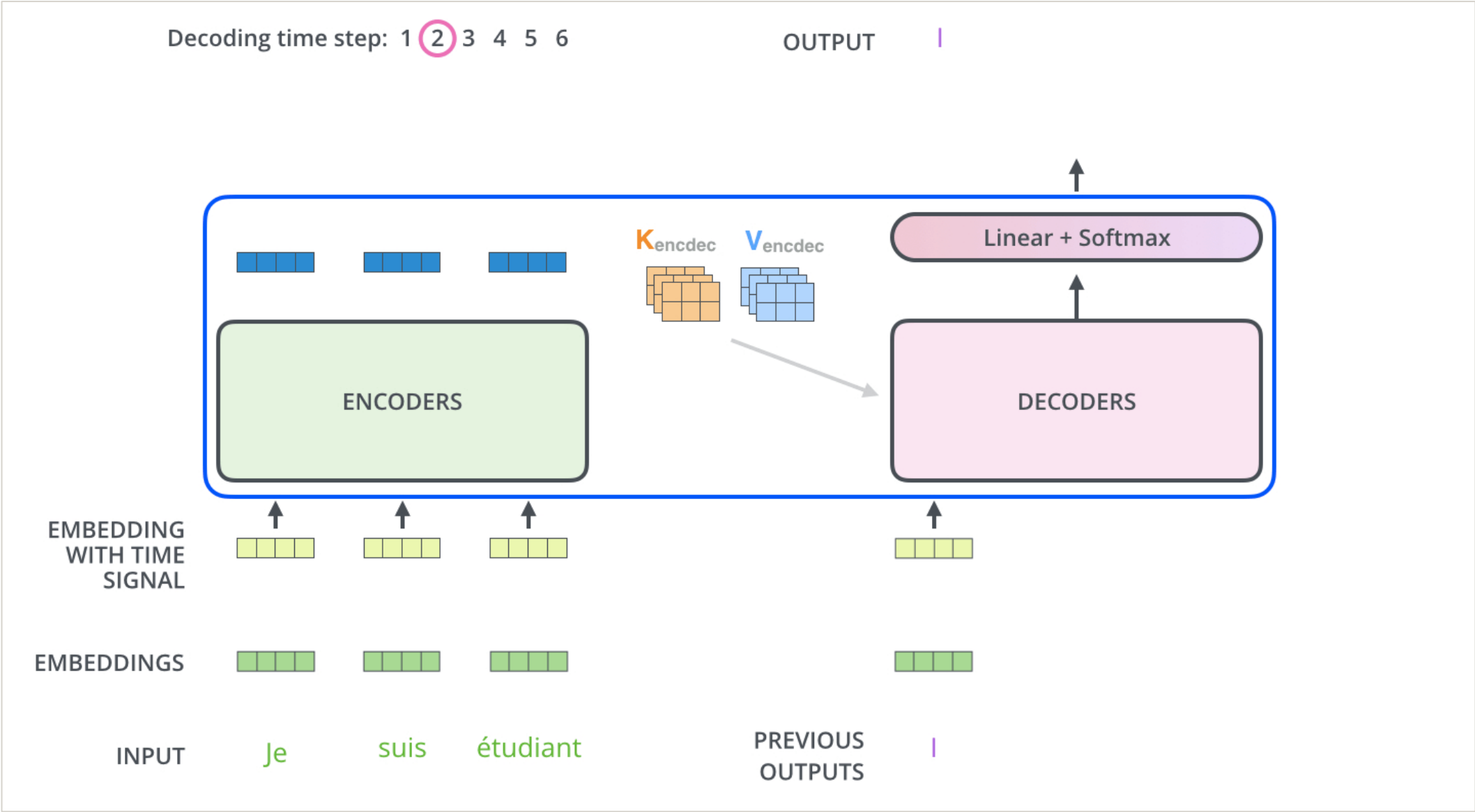
- **Input Processing:**
 - The encoder processes the input sequence, producing intermediate representations.
- **Encoder Output Transformation:**
 - The output of the top encoder is transformed into attention vectors K and V.
- **Decoder Self-Attention:**
 - Each decoder layer performs self-attention, allowing it to attend to previous positions in the output sequence.
- **Encoder-Decoder Attention:**
 - The decoder utilizes the attention vectors K and V from the encoder to focus on appropriate places in the input sequence.
- **Feed-Forward Neural Networks:**
 - After attention mechanisms, the data is passed through feed-forward neural networks for further processing.



Overview of Transformer Model



Overview of Transformer Model



Advantages of Transformer:

Scalability:

- Efficiently handles large datasets and complex tasks.

Transfer Learning:

- Pre-trained models like BERT and GPT facilitate adaptation to new tasks with minimal data.

Multimodal:

- Capable of processing and integrating various data types, including text, image, and audio.

Flexible Architecture:

- Encoder Only: e.g., BERT for understanding context in text.
- Decoder Only: e.g., GPT for generating coherent text based on prompts.

Rich Ecosystem:

- Supports extensive libraries like Hugging Face for easy model deployment and fine-tuning.
- Abundant resources like videos and blogs for learning and development.

Disadvantages:

- High Computation(GPU): it requires high computational resources
- Data (LLMs)
- Overfitting
- Energy Consumption
- Ethical Concerns

Real world applications of Transformer:

ChatGPT, DALL-E 2, AlphaFold, OpenAI Codex

Can we use a Transformer as-is for images?

Using a Transformer as-is?

 **Transformer Self-Attention** – each token attends all other tokens

 **Feeding an image to transformer** – each pixel attends all other pixels

256 X 256
image

512 X 512
image

Input sequence



65k pixels

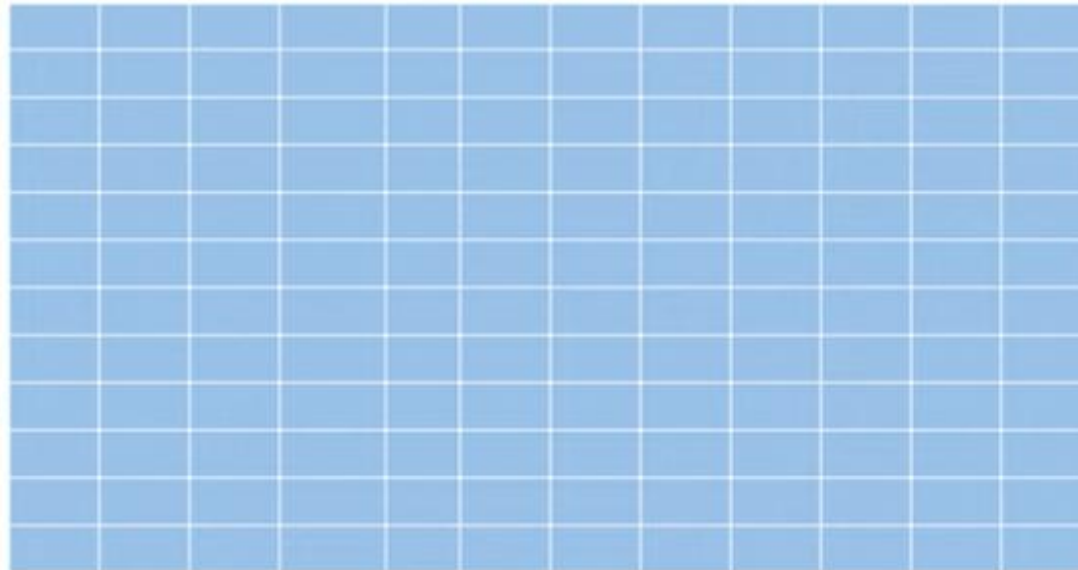
262k pixels



Attention matrix



**Quadratic dependency
on the sequence length**



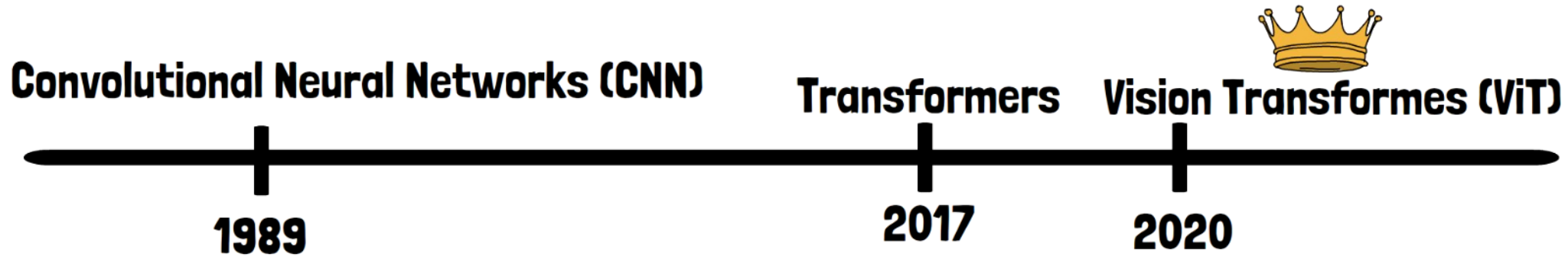
65k X 65k
attention
matrix

262k X 262k
attention
matrix



 **Handling images as-is results in too long sequence lengths**

Evolution of Dominant Architectures in Computer Vision



❑ Rise of CNNs (1989 - 2020)

- Convolutional Neural Networks (CNNs) dominated computer vision.
- Invented in 1989 by Yann LeCun and Yoshua Bengio.
- Excelled in image classification, object detection, and segmentation.

❑ Advent of Vision Transformers (ViTs) (2020 - Present)

- Transformers revolutionized NLP in 2017, introduced by Google.
- Successfully adapted to computer vision in 2020 with Vision Transformers (ViT) by Google.
- Became the backbone of top models like DINOv2, DeiT, and CLIP.

❑ Key Paper: "An Image Is Worth 16x16 Words: Transformers For Image Recognition At Scale"

- Foundation of ViT architecture and its application in image recognition.

Vision Transformers (ViTs)

Vision Transformers (ViTs)

- It analyze images by dividing them into smaller patches and leveraging self-attention mechanisms.
- Vision Transformers apply the Transformer architecture, which was initially developed for NLP, to image data.
- Introduced by the paper “**An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale**” by Dosovitskiy et al. (2020).
- It significantly reducing the sequence length.

Key Stages in Vision Transformers

❑ Breaking the Image into Patches:

Split the image into smaller patches instead of feeding pixel sequences directly.

❑ Adapting Patches for Transformer Input:

Use a trainable linear projection layer to convert patches into vectors suitable for transformers.

❑ Adding Positional Embeddings:

Incorporate positional embeddings to retain patch location information within the image.

❑ Processing with Transformer:

Feed patch embeddings to the transformer along with a special learnable embedding to grasp global information about the image, useful for various tasks including classification.

Vision Transformers (ViTs)

Key Concepts:

Image Patching:

- Instead of processing an image as a whole, the image is divided into fixed-size patches (e.g., 16x16 pixels).
- Each patch is treated similarly to a token in NLP.

Linear Embedding:

- Each image patch is flattened into a 1D vector.
- These vectors are then linearly embedded into a higher-dimensional space, producing patch embeddings.

Positional Encoding:

- Since Transformers do not have a built-in notion of order, positional encodings are added to the patch embeddings to retain spatial information.

Transformer Encoder:

- The sequence of embedded patches, along with positional encodings, is fed into the Transformer encoder.
- The encoder processes the sequence through self-attention mechanisms and feed-forward neural networks.

Classification Token:

- A special classification token ([CLS] token) is prepended to the sequence of patch embeddings.
- The output corresponding to this token is used for image-level predictions, such as classification.

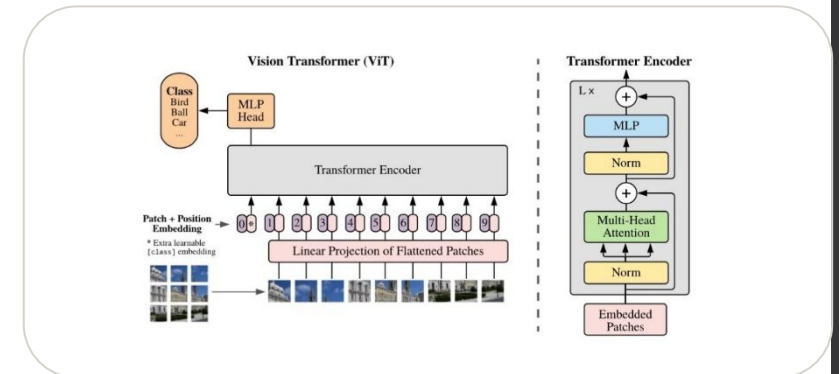
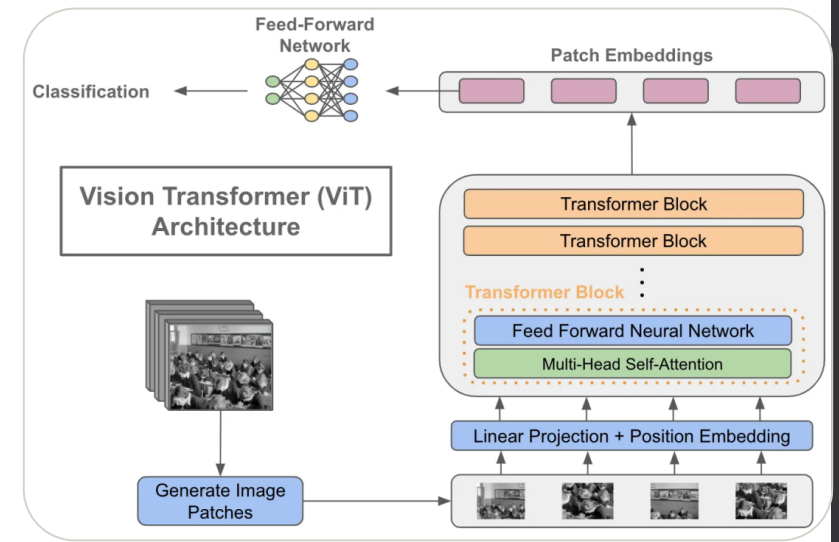


Figure 1: Self-attention from a Vision Transformer with 8 x 8 patches trained with no supervision. We look at the self-attention of the [CLS] token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentation.

Vision Transformers (ViTs)

Advantages:

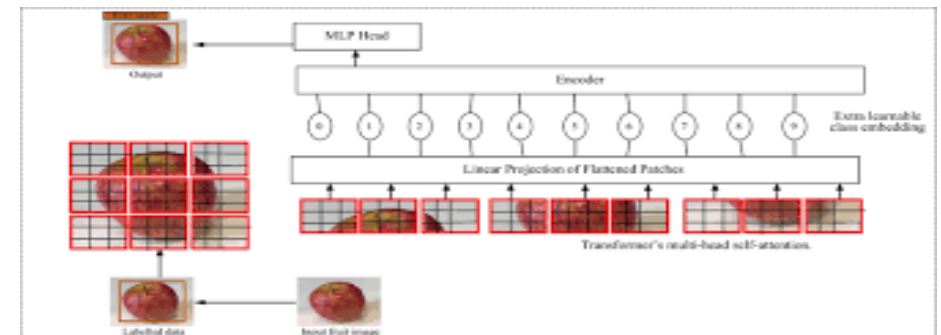
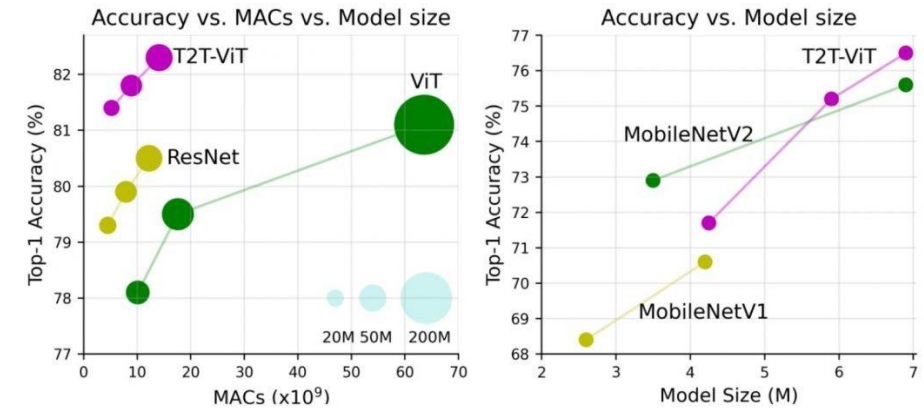
- **Scalability:** ViTs can leverage large-scale pre-training on extensive datasets, similar to NLP models.
- **Flexibility:** They can be easily adapted for various vision tasks, including image classification, object detection, and segmentation.
- **Performance:** When pre-trained on large datasets, ViTs can achieve performance comparable to or exceeding that of traditional convolutional neural networks (CNNs).

Challenges:

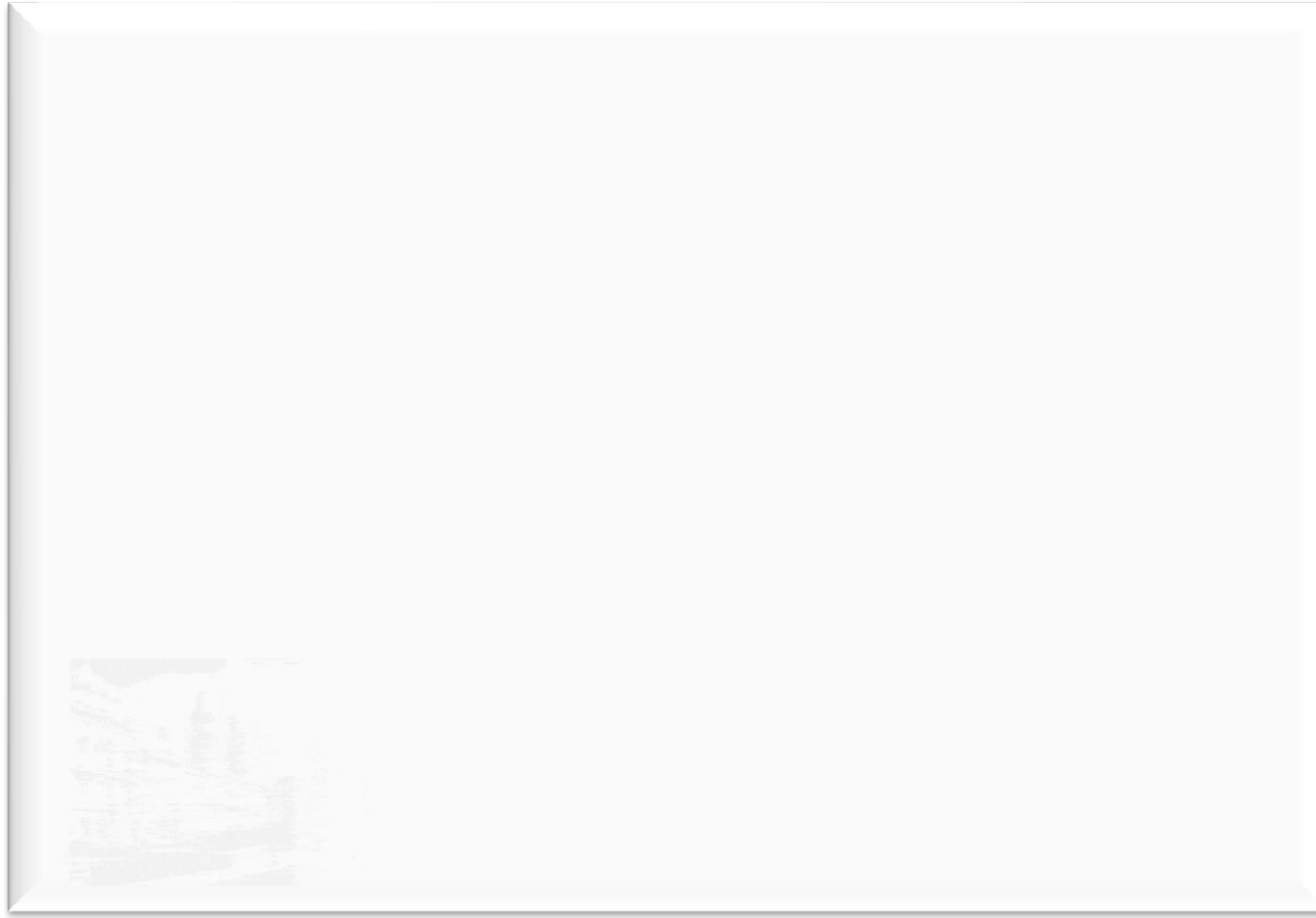
- **Data Efficiency:** ViTs require a large amount of training data to perform well, as they lack the inductive biases present in CNNs.
- **Computationally Intensive:** They demand significant computational resources for training due to the self-attention mechanism's complexity.

Applications:

- **Image Classification:** ViTs have been successfully applied to image classification tasks, achieving state-of-the-art results on benchmarks like ImageNet.
- **Object Detection:** ViTs can be adapted for object detection tasks, providing competitive performance.
- **Image Segmentation:** ViTs can also be used for segmentation tasks, leveraging their ability to model long-range dependencies.



Vision transformer architecture



Vision Transformers vs. CNN?

ViT differs from Convolutional Neural Networks (CNNs) in several key aspects:

❑ Input Representation:

CNNs: Process raw pixel values directly.

ViT: Divides input image into patches, transforming them into tokens.

❑ Processing Mechanism:

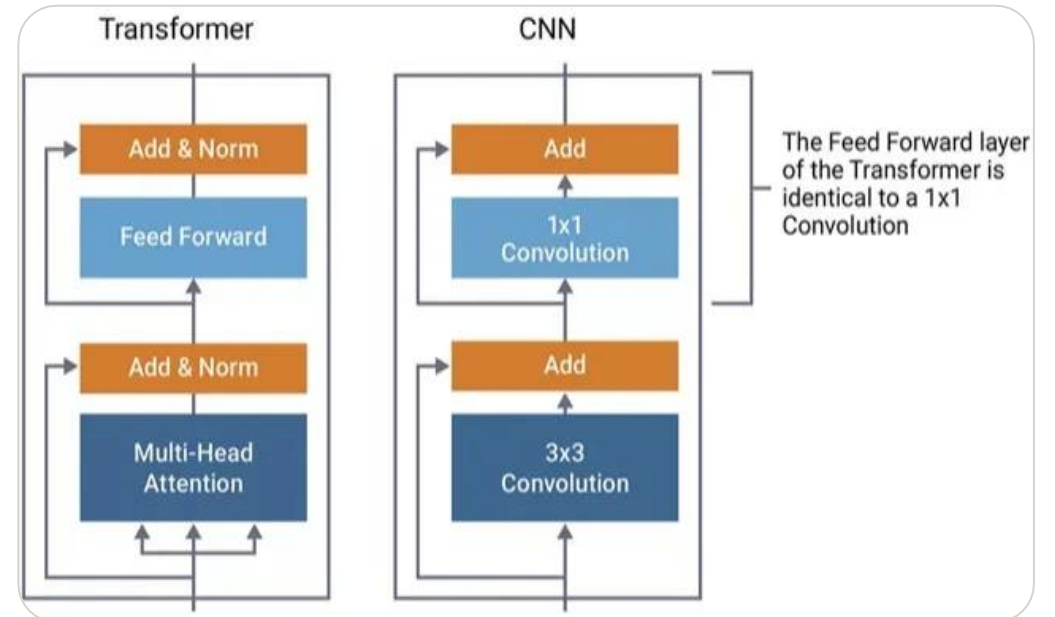
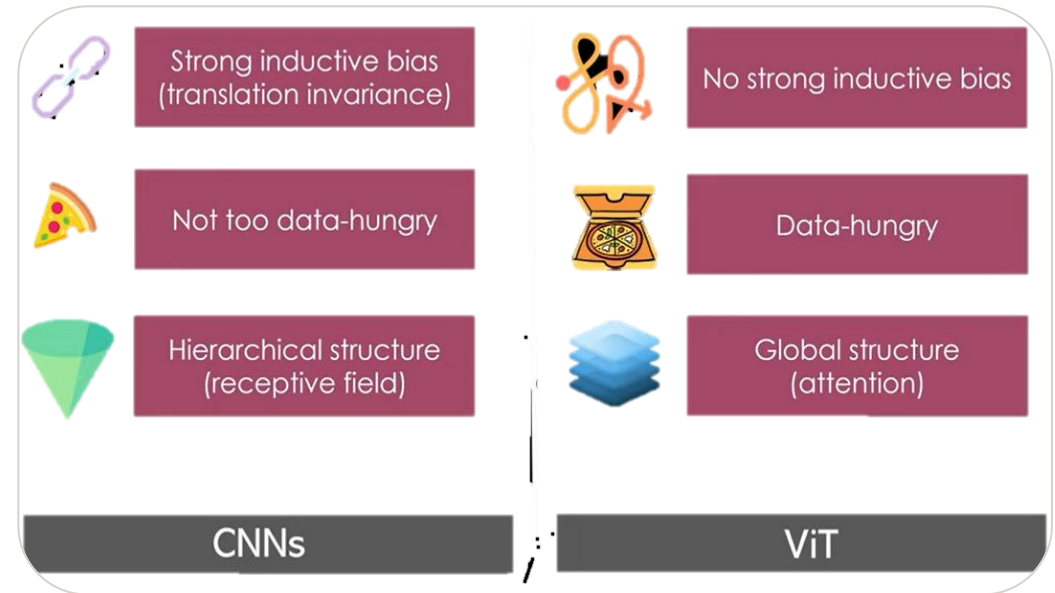
CNNs: Use convolutional and pooling layers for hierarchical feature capture.

ViT: Employs self-attention mechanisms for relationships among all patches.

❑ Global Context:

CNNs: Rely on pooling layers for coarse global information.

ViT: Captures global context through self-attention.



References & Links

Transformer Paper:

- https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

Blog:

- <https://jalammar.github.io/illustrated-transformer/>

Vision Transformer Paper:

- <https://arxiv.org/abs/2010.11929>

Blogs:

- <https://aipapersacademy.com/vision-transformers/>
- <https://semiengineering.com/achieving-greater-accuracy-in-real-time-vision-processing-with-transformers/>

Thank You